4.4 Theory of computation

4.4.1 Abstraction and automation

4.4.1.1 Problem-solving

Content	Additional information
Be able to develop solutions to simple logic problems.	
Be able to check solutions to simple logic problems.	

4.4.1.2 Following and writing algorithms

Content	Additional information
Understand the term algorithm.	A sequence of steps that can be followed to complete a task and that always terminates.
Be able to express the solution to a simple problem as an algorithm using pseudo-code, with the standard constructs: • sequence • assignment • selection • iteration.	
Be able to hand-trace algorithms.	
Be able to convert an algorithm from pseudo-code into high level language program code.	

Content	Additional information
Be able to articulate how a program works, arguing for its correctness and its efficiency using logical reasoning, test data and user feedback.	

4.4.1.3 Abstraction

Content	Additional information
Be familiar with the concept of abstraction as used in computations and know that: • representational abstraction is a representation arrived at by removing unnecessary details • abstraction by generalisation or categorisation is a grouping by common characteristics to arrive at a hierarchical relationship of the 'is a kind of' type.	

4.4.1.4 Information hiding

Content	Additional information
Be familiar with the process of hiding all details of an object that do not contribute to its essential characteristics.	

4.4.1.5 Procedural abstraction

Content	Additional information
Know that procedural abstraction represents a computational method.	The result of abstracting away the actual values used in any particular computation is a computational pattern or computational method - a procedure.

4.4.1.6 Functional abstraction

Content	Additional information
Know that for functional abstraction the particular computation method is hidden.	The result of a procedural abstraction is a procedure, not a function. To get a function requires yet another abstraction, which disregards the particular computation method. This is functional abstraction.

4.4.1.7 Data abstraction

Content	Additional information
Know that details of how data are actually represented are hidden, allowing new kinds of data objects to be constructed from previously defined types of data objects.	Data abstraction is a methodology that enables us to isolate how a compound data object is used from the details of how it is constructed. For example, a stack could be implemented as an array and a pointer for top of stack.

4.4.1.8 Problem abstraction/reduction

Content	Additional information
Know that details are removed until the problem is represented in a way that is possible to solve, because the problem reduces to one that has already been solved.	

4.4.1.9 Decomposition

Content	Additional information
Know that procedural decomposition means breaking a problem into a number of sub-problems, so that each sub-problem accomplishes an identifiable task, which might itself be further subdivided.	

4.4.1.10 Composition

Content	Additional information
Know how to build a composition abstraction by combining procedures to form compound procedures.	
Know how to build data abstractions by combining data objects to form compound data, for example tree data structure.	

4.4.1.11 Automation

Content	Additional information
Understand that automation requires putting models (abstraction of real world objects/ phenomena) into action to solve problems. This is achieved by: • creating algorithms • implementing the algorithms in program code (instructions) • implementing the models in data structures • executing the code.	Computer science is about building clean abstract models (abstractions) of messy, noisy, real world objects or phenomena. Computer scientists have to choose what to include in models and what to discard, to determine the minimum amount of detail necessary to model in order to solve a given problem to the required degree of accuracy. Computer science deals with putting the models into action to solve problems. This involves creating algorithms for performing actions on, and with, the data that has been modelled.